Building an Agentic AI Application on AWS

A Complete Learning Guide for Aspiring AI Engineers

Who This Is For

This documentation is written for two kinds of readers:

- **Beginner software engineers** who know some Python but haven't built AI applications or worked with cloud APIs before
- **Experienced engineers** who are comfortable with code but are new to the world of AI agents, LLMs, and tool use

By the end of this guide, you will understand not just *what* this project does, but *why* every design decision was made — and you'll have the mental model to build similar AI-powered applications yourself.

Table of Contents

1. What Are We Building and Why?
2. Key Technologies and Concepts
3. Project Structure Overview
4. Layer 1 — Configuration: Connecting to AWS
5. Layer 2 — AI Building Blocks: Tools and Prompts
6. Layer 3 — The AI Agent: The Brain of the Application
7. Layer 4 — The API Server: Exposing the Agent over HTTP
8. Layer 5 — The Frontend: The Chat Interface
9. End-to-End Request Flow
10. Sample Interactions
11. Core AI Engineering Patterns
12. What to Learn Next

---

1. What Are We Building and Why?

The Problem

Managing cloud infrastructure on AWS typically requires you to:

- Know the AWS Console navigation deeply
- Write infrastructure-as-code (CloudFormation, Terraform)
- Remember dozens of CLI commands and API parameters

This is a steep learning curve and slows down teams.

The Solution

This project lets anyone manage AWS resources using **plain English**:

```
You:  "Show me all my S3 buckets"
AI:   "You have 3 S3 buckets: my-photos, backups, and static-assets."

You:  "Create an S3 bucket for storing project logs"
AI:   "What would you like to name the bucket? S3 bucket names must be
globally unique."

You:  "call it project-logs-2024-sanjay"
AI:   "Creating the bucket now... ✅ Successfully initiated creation of
'project-logs-2024-sanjay'"

You:  "Find out errors from my lambda function in the past 24 hours"

AI:   "I'd be happy to help you find errors from your Lambda function in
the past
       24 hours. However, I need one piece of information:

       What is the name of your Lambda function?"
```

The AI figures out which AWS API to call, with what parameters, and explains the result in human language. This is called an **AI agent**.

Why This Project Is a Great Learning Path

This application is small enough to read completely in one sitting, but rich enough to teach you the most important concepts in modern AI engineering:

- How to give an AI model real-world capabilities (tool use)
- How to control AI behavior through prompting
- How to build a full-stack application with a Python backend and chat UI
- How to integrate with real cloud APIs (AWS)

---

2. Key Technologies and Concepts

Before diving into code, let's build a mental model of each technology used.

Python

The entire backend and frontend are written in Python. You should be comfortable with classes, functions, and importing modules.

Claude (by Anthropic)

Claude is an AI language model — similar to ChatGPT. In this project, we use Claude for two things:

1. To **understand the user's intent** and decide which AWS operation to run
2. To **explain the result** of the AWS operation in plain English

We interact with Claude through the Anthropic Python SDK by making API calls — the same way you'd call any web API.

FastAPI

FastAPI is a modern Python framework for building web APIs. Think of it as the "server" that listens for requests from the chat interface and routes them to the AI agent. It's the intermediary between the frontend (what the user sees) and the backend logic.

Streamlit

Streamlit is a Python library for building simple web UIs — especially popular for data science and AI demos. It lets you build a functional chat interface in under 50 lines of Python, without writing any HTML or JavaScript.

boto3 (AWS SDK for Python)

boto3 is the official AWS library for Python. It lets your Python code talk to AWS services — create S3 buckets, query CloudWatch logs, etc. — without manually making HTTP requests.

AWS Cloud Control API

Instead of having a separate boto3 client for every AWS service (one for S3, one for EC2, one for RDS...), Cloud Control API is a **unified interface** for managing almost any AWS resource type. You specify the resource using a type string like `AWS::S3::Bucket` and the operation (`create`, `list`, `read`, etc.), and it handles the rest.

What Is an "Agent"?

This is the most important concept in this project. An **AI agent** is a system where:

1. A user sends a message
2. An AI model **reasons** about what to do
3. The AI calls a **tool** (a real function in your code)
4. The result is fed back to the AI
5. The AI **responds** in natural language

The key insight: **the AI doesn't directly run code**. It tells your application *what to run*, and your application executes it. The AI acts as the brain; your code acts as the hands.

```
User message
    ↓
AI reasons: "I should list S3 buckets"
    ↓
AI returns: { "tool": "aws_cloud_control", "operation": "list",
"resource_type": "AWS::S3::Bucket" }
    ↓
Your code calls the real AWS API
    ↓
AWS returns raw JSON data
```

```
    ↓
AI translates raw data → plain English reply
    ↓
User sees the answer
```

---

3. Project Structure Overview

The project is organized into clear layers, each with a single responsibility:

```
agentic-aws-resource-management/
│
├── frontend/
│   └── chat.py                ← What the user sees (Streamlit chat UI)
│
├── backend/
│   ├── api/
│   │   └── main.py            ← HTTP entry point (FastAPI server)
│   │
│   ├── config/
│   │   ├── aws_config.py      ← AWS credentials and connection setup
│   │   └── prompt.py          ← Instructions that shape the AI's behavior
│   │
│   ├── core/
│   │   ├── aws_agent.py       ← The AI agent (most important file)
│   │   └── processor.py       ← Connects the API layer to the agent
│   │
│   └── tools/
│       └── tools.json         ← Defines what the AI is allowed to do
│
├── requirements.txt           ← Python package dependencies
└── .env                       ← Your secret credentials (never commit this)
```

How to read this: Data flows from `frontend/chat.py` → `backend/api/main.py` → `backend/core/aws_agent.py` → AWS and Anthropic APIs → back up the same chain. We'll walk through each layer from the bottom up, building understanding as we go.

---

4. Layer 1 — Configuration: Connecting to AWS

File: `backend/config/aws_config.py`

Before the AI agent can do anything, it needs authenticated access to AWS. This file handles that.

Environment Variables and the `.env` File

Credentials like AWS access keys and API keys must **never** be hardcoded in your source code. If you push code to GitHub with a hardcoded key, that key is compromised — bots scan public repos constantly.

The standard practice is to store secrets in a `.env` file (which is added to `.gitignore` so it's never committed) and load them at runtime:

```
# .env file
ANTHROPIC_API_KEY=sk-ant-...
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
AWS_DEFAULT_REGION=us-east-1
```

The `python-dotenv` library reads this file and makes these values available via `os.getenv()`.

The AWSConfig Class

```
import boto3
import os
from dotenv import load_dotenv

load_dotenv()  # reads the .env file into environment variables

class AWSConfig:
    def __init__(self):
        self.access_key = os.getenv('AWS_ACCESS_KEY_ID')
        self.secret_key = os.getenv('AWS_SECRET_ACCESS_KEY')
        self.region     = os.getenv('AWS_DEFAULT_REGION', 'us-east-1')

    def get_session(self):
        return boto3.Session(
            aws_access_key_id=self.access_key,
            aws_secret_access_key=self.secret_key,
            region_name=self.region
        )

    def validate_connection(self):
        try:
            sts = self.get_session().client('sts')
            identity = sts.get_caller_identity()
            print(f"✅ AWS Connection successful! Account:
{identity['Account']}")
            return True
        except Exception as e:
            print(f"❌ AWS Connection failed: {e}")
            return False
```

What is a boto3 Session? A `boto3.Session` is an authenticated AWS context. From a session, you can create service clients — for example, `session.client('s3')` gives you an S3 client, `session.client('cloudcontrol')` gives you a Cloud Control client. Credentials are attached once to the session and reused across all clients.

What is STS `get_caller_identity`? AWS Security Token Service (STS) has an endpoint that simply returns your account ID if your credentials are valid. It's the lightest-weight "am I authenticated?" check available — equivalent to a `ping` for AWS credentials. This is called at startup to fail fast rather than failing mid-request.

---

5. Layer 2 — AI Building Blocks: Tools and Prompts

Two files define *how* the AI behaves and *what it is allowed to do*. These are the most important design artifacts in an AI application.

5.1 Tool Definitions — `backend/tools/tools.json`

What is "tool use"?

Modern AI models like Claude can be given a list of tools — structured function definitions — alongside the user's message. The model reads these definitions and, when appropriate, responds not with text but with a structured call to one of the tools. Your application then runs the actual function and optionally sends the result back to the AI for a follow-up response.

This is how an AI moves from being a chatbot (that only talks) to an agent (that takes real actions).

This project defines two tools:

```
[
    {
        "name": "aws_cloud_control",
        "description": "Execute AWS Cloud Control API operations for
infrastructure management",
        "input_schema": {
            "type": "object",
            "properties": {
                "operation": {
                    "type": "string",
                    "enum": ["create", "read", "update", "delete", "list"],
                    "description": "The operation to perform"
                },
                "resource_type": {
                    "type": "string",
                    "description": "AWS resource type (e.g.,
AWS::S3::Bucket, AWS::EC2::Instance)"
                },
                "identifier": {
                    "type": "string",
                    "description": "Resource identifier for
read/update/delete operations"
                },
                "properties": {
                    "type": "object",
                    "description": "Resource properties for create/update
operations"
                },
```

```
                "region": {
                    "type": "string",
                    "default": "us-east-1"
                }
            },
            "required": ["operation", "resource_type"]
        }
    },
    {
        "name": "cloudwatch_logs",
        "description": "Query CloudWatch Logs for Lambda function error
logs",
        "input_schema": {
            "type": "object",
            "properties": {
                "function_name": {
                    "type": "string",
                    "description": "Name of the Lambda function to check
for errors"
                },
                "hours_back": {
                    "type": "integer",
                    "default": 1,
                    "description": "Number of hours to look back for
errors"
                }
            },
            "required": ["function_name"]
        }
    }
]
```

Why write this in JSON and not in Python code? Separating the tool schema from the application logic makes it easy to add new tools later without touching any Python. The AI agent loads this file at runtime, so a new tool is just a new JSON object in this file.

Key insight — the `description` field is critical: Claude doesn't look at the tool's name to decide whether to use it — it reads the `description`. A vague description leads to wrong tool selection. A specific, accurate description leads to reliable behavior. Writing good tool descriptions is a skill at the heart of AI engineering.

The `input_schema` is a JSON Schema: JSON Schema is a standard format for defining the shape of a JSON object. The `"required"` array tells Claude which fields must always be provided. The `"enum"` constraint on `operation` limits the AI to only valid operation types, preventing hallucinated values.

5.2 System Prompts — `backend/config/prompt.py`

What is a system prompt?

Every conversation with an AI model starts with a hidden set of instructions — the *system prompt*. This is how you configure the AI's personality, rules, and behavioral constraints. Users never see this; the model

always does.

```
SYSTEM_PROMPT = """You are an AWS infrastructure management and Monitoring
agent.

CRITICAL RULES - ALWAYS FOLLOW:
1. NEVER assume missing required information
2. ALWAYS ask for clarification when key details are missing
3. For S3 buckets, ALWAYS ask for the bucket name - never generate one
4. For EC2 instances, ALWAYS ask for instance type, key pair name
5. For any resource creation, confirm all required parameters first

When a user requests resource creation:
1. First, identify what information you need
2. Ask specific questions for missing details
3. Only proceed with creation after you have ALL required information
4. If you must suggest defaults, present them as options for user approval

Current AWS region: us-east-1"""
```

Why are rules written in ALL CAPS? Claude (and most LLMs) pay more attention to emphasized text. Using "NEVER" and "ALWAYS" is a common prompting technique to establish hard constraints that the model should treat as inviolable rather than guidelines.

Why not let Claude generate bucket names on its own? This is a deliberate safety decision. S3 bucket names are globally unique — a generated name might clash with an existing bucket, or worse, the user might not recognize the resource later. Always confirming names before creation prevents mistakes that are hard to reverse in a live cloud account.

The second prompt is used *after* an AWS operation runs:

```
SUMMARY_PROMPT = """
The user asked: "{user_question}"

I executed the {tool_name} tool and got these results:
{tool_result}

Please provide a natural language summary of these results.
Be helpful and explain what the user should know.
Keep it conversational and clear.
"""
```

Why a second prompt? AWS API responses are raw JSON full of request tokens, ARNs, and status codes that are meaningless to most users. Rather than display this directly, we use a second AI call to translate it. This is called the **Summarizer pattern** — using a cheap, fast model call to make raw data human-readable.

---

6. Layer 3 — The AI Agent: The Brain of the Application

File: `backend/core/aws_agent.py`

This is the most important file in the project. It orchestrates the entire interaction between the user, Claude, and AWS.

Initialization

```
class AWSAgenticAgent:
    def __init__(self):
        self.client = anthropic.Anthropic(
            api_key=os.getenv('ANTHROPIC_API_KEY')
        )
        self.aws_config = AWSConfig()
        self._test_aws_connection()
```

On startup, two connections are established:

- `anthropic.Anthropic` — the Claude AI client. All calls to Claude go through this object.
- `AWSConfig` — the AWS session manager we covered in Layer 1.

AWS connection is validated immediately. If credentials are wrong, the app fails at startup with a clear error — not mid-conversation with an ambiguous message.

The Core Agent Loop — `process_request()`

This is where the magic happens. Read this carefully:

```
def process_request(self, user_input: str, history: List[Dict]) -> str:

    # Step 1: Add the user's new message to the conversation history
    history.append({"role": "user", "content": user_input})

    # Step 2: Send the full conversation to Claude, along with the tools it
can use
    response = self.client.messages.create(
        model="claude-haiku-4-5",
        max_tokens=2000,
        system=self._build_system_prompt(),   # the rules from prompt.py
        messages=history,                      # the entire conversation so
far
        tools=self._format_tools()             # the tool definitions from
tools.json
    )

    response_content = ""

    # Step 3: Claude's response may contain text, tool calls, or both
```

```
    for content_block in response.content:

        if content_block.type == "text":
            # Claude is asking a question or giving an explanation — no
action needed
            response_content += content_block.text

        elif content_block.type == "tool_use":
            # Claude decided to call a tool
            tool_name  = content_block.name    # e.g. "aws_cloud_control"
            tool_input = content_block.input   # e.g. {"operation": "list",
"resource_type": "AWS::S3::Bucket"}

            if tool_name == "aws_cloud_control":
                result  = self._execute_aws_operation(**tool_input)
                summary = self._generate_summary(tool_name, result,
user_input)
                response_content += f"\n\n🤖 AI Summary:\n{summary}"

            elif tool_name == "cloudwatch_logs":
                result  = self._query_cloudwatch_logs(**tool_input)
                summary = self._generate_summary(tool_name, result,
user_input)
                response_content += f"\n\n🤖 AI Summary:\n{summary}"

    # Step 4: Add Claude's reply to the history for future turns
    history.append({"role": "assistant", "content": response_content})

    return response_content
```

Let's unpack each step.

Step 1 — History is the memory

The `history` list contains every message in the conversation:

```
[
    {"role": "user",      "content": "Show me my S3 buckets"},
    {"role": "assistant", "content": "You have 3 buckets: ..."},
    {"role": "user",      "content": "Delete the backups one"}
]
```

This entire list is sent to Claude on every request. Because Claude sees the full history, it knows "the backups one" refers to `data-backup-bucket` from two messages earlier. **This is how conversational memory works** in LLM applications — there's no magic database of memory. The conversation is just reconstructed from scratch on every call.

Step 2 — The Claude API call

```
response = self.client.messages.create(
    model="claude-haiku-4-5",   # fast and cost-efficient model
    max_tokens=2000,             # maximum length of the response
    system=SYSTEM_PROMPT,        # the hidden rules
    messages=history,            # the conversation
    tools=tool_definitions       # what Claude is allowed to do
)
```

`max_tokens` limits how long Claude's response can be. This controls cost and prevents runaway responses.

Why `claude-haiku-4-5` and not a more powerful model? Haiku is Anthropic's fastest and cheapest model. For the tasks here — picking a tool and summarizing a result — it is more than capable. Using a more powerful model like Opus would add latency and cost without improving the outcome. Matching model capability to task complexity is an important cost-optimization skill in AI engineering.

Step 3 — Handling `content_block.type`

Claude's response is not a plain string — it's a list of *content blocks*. Each block has a type:

- `"text"` — Claude is replying in natural language. This happens when Claude needs more information or wants to explain something.
- `"tool_use"` — Claude has decided to call a tool. This block contains `name` (which tool) and `input` (the arguments as a dictionary).

The `for` loop handles both. In a real conversation, Claude may return both types in the same response (e.g., "I'll create that for you now" as text, plus a tool call).

Step 4 — Appending the reply to history

After generating the response, it's added back to `history`. On the *next* request, this updated history (now including the assistant's reply) will be sent back to Claude so it maintains context.

Executing AWS Operations — `_execute_aws_operation()`

```
def _execute_aws_operation(self, operation, resource_type,
                           identifier=None, properties=None, region="us-
east-1"):

    cloudcontrol = self.aws_config.get_session().client('cloudcontrol',
region_name=region)

    if operation == "create":
        response = cloudcontrol.create_resource(
            TypeName=resource_type,                              # e.g.
"AWS::S3::Bucket"
            DesiredState=json.dumps(properties) if properties else "{}"
        )
        request_token = response['ProgressEvent']['RequestToken']
```

```
            return {
                "status": "success",
                "request_token": request_token,
                "message": f"Creating {resource_type}..."
            }

        elif operation == "list":
            response = cloudcontrol.list_resources(TypeName=resource_type)
            resources = response.get('ResourceDescriptions', [])
            return {
                "status": "success",
                "count": len(resources),
                "resources": [json.loads(r['Properties']) for r in
resources[:5]]
            }

        elif operation == "read" and identifier:
            response = cloudcontrol.get_resource(TypeName=resource_type,
Identifier=identifier)
            return {
                "status": "success",
                "properties": json.loads(response['ResourceDescription']
['Properties'])
            }
```

Why Cloud Control API instead of individual service clients?

Without Cloud Control, you would need:

- `session.client('s3')` to manage S3
- `session.client('ec2')` to manage EC2
- `session.client('rds')` to manage RDS
- etc.

And each would have its own method names, parameter shapes, and response formats. Cloud Control unifies all of these behind one interface — `create_resource`, `list_resources`, `get_resource` — with the resource type as a string. Adding support for a new AWS resource type requires **no code change**, only a mention in the prompts.

Important: Cloud Control `create` is asynchronous

When you call `create_resource`, AWS does *not* return the created resource. It returns a `RequestToken` — a reference you can use to check the operation's progress later. The resource may take several seconds to minutes to actually exist. This is why the success response contains a token, not the bucket name or ARN. Users should wait a moment after creation before trying to access the new resource.

Querying CloudWatch Logs — `_query_cloudwatch_logs()`

```
def _query_cloudwatch_logs(self, function_name, hours_back=1):

    logs_client = self.aws_config.get_session().client('logs')

    end_time   = datetime.now()
    start_time = end_time - timedelta(hours=hours_back)

    response = logs_client.filter_log_events(
        logGroupName=f"/aws/lambda/{function_name}",  # Lambda auto-creates
this log group
        startTime=int(start_time.timestamp() * 1000), # CloudWatch uses
milliseconds
        endTime=int(end_time.timestamp() * 1000),
        filterPattern="ERROR"
    )

    return {
        "status": "success",
        "function_name": function_name,
        "error_count": len(response['events']),
        "error_logs": response['events'][:10]         # cap at 10 to keep
response manageable
    }
```

Why `/aws/lambda/{function_name}`? AWS Lambda automatically writes all function output (print statements, errors) to CloudWatch Logs under a log group named `/aws/lambda/<your-function-name>`. This naming convention is enforced by AWS, so we can derive the log group path directly from the function name without any lookup.

Why `timestamp * 1000`? Python's `datetime.timestamp()` returns seconds since Unix epoch. CloudWatch Logs expects milliseconds. This off-by-1000 is a very common bug when working with AWS time parameters — always check the unit expected by the API.

Generating Human-Readable Summaries — `_generate_summary()`

```
def _generate_summary(self, tool_name, tool_result, user_question):

    summary_prompt = SUMMARY_PROMPT.format(
        tool_name=tool_name,
        tool_result=json.dumps(tool_result, indent=2),
        user_question=user_question
    )

    response = self.client.messages.create(
        model="claude-haiku-4-5",
        max_tokens=500,
        messages=[{"role": "user", "content": summary_prompt}]
    )
```

```
    return response.content[0].text
```

This makes a **second** Claude API call. The first call decided what to do; this call explains what happened. The raw AWS response (full of tokens and ARN strings) is passed in, and Claude returns a friendly sentence like "Your S3 bucket was successfully created and should be available in a few seconds."

This two-call pattern is deliberate. Mixing "decide what to do" and "explain what happened" in a single prompt makes both tasks harder. Separating them into focused prompts produces better results for each.

---

7. Layer 4 — The API Server: Exposing the Agent over HTTP

Files: `backend/api/main.py`, `backend/core/processor.py`

Why Do We Need an API Server?

The AI agent is Python code. The chat interface is a Streamlit app (also Python, but running in its own process on a different port). They can't import each other directly — they communicate over HTTP, the same protocol your browser uses to load websites.

FastAPI creates a server that listens for HTTP requests on port 8000 and routes them to the right handler function.

`backend/api/main.py`

```
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from backend.core.processor import process_request

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],    # allow requests from any origin (Streamlit on
port 8501)
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def root():
    return {"message": "FastAPI is running"}

@app.post("/chat")
async def chat(request: Request):
    data       = await request.json()
    user_input = data.get("message", "")
    history    = data.get("history", [])

    response = process_request(user_input, history)
```

```
    return {
        "response": response,
        "updated_history": history
    }
```

What is CORS?

CORS (Cross-Origin Resource Sharing) is a browser security feature that blocks web pages from making requests to a different domain or port than the one they came from. The Streamlit UI runs on port `8501` and the FastAPI backend runs on port `8000` — different ports count as different origins. Without the CORS middleware, the browser would block the Streamlit app from calling the backend.

`allow_origins=["*"]` allows any origin. This is fine for local development but should be restricted to specific origins (e.g., your deployed frontend URL) in production.

The history round-trip

Notice that `history` is received from the frontend, mutated by `process_request`, and then returned as `updated_history`. The backend is completely stateless — it doesn't remember any conversation. Every conversation's history lives in the frontend's session state and travels with each request. This design makes the backend simple and easy to scale.

backend/core/processor.py

```
from backend.core.aws_agent import AWSAgenticAgent
from typing import List, Dict

def process_request(user_message: str, history: List[Dict]) -> str:
    agent = AWSAgenticAgent()
    return agent.process_request(user_message, history=history)
```

This is intentionally thin. Its only job is to instantiate the agent and delegate. This separation means:

- `main.py` handles HTTP concerns only — no AWS or AI logic
- `aws_agent.py` handles AI and AWS logic only — no HTTP concepts
- `processor.py` is the seam between them

Clean separation of concerns makes code easier to test, modify, and reason about.

---

8. Layer 5 — The Frontend: The Chat Interface

File: `frontend/chat.py`

```
import streamlit as st
import requests
```

```
st.set_page_config(page_title="Agentic AWS Chatbot", layout="centered")
st.title("Agentic AWS Chatbot")

# Session state persists across re-renders — this is where conversation
history lives
if "messages" not in st.session_state:
    st.session_state["messages"] = []

# Render every previous message in the conversation
for msg in st.session_state["messages"]:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

# The chat input box at the bottom of the screen
user_input = st.chat_input("Say something...")

if user_input:
    # Immediately render the user's message
    with st.chat_message("user"):
        st.markdown(user_input)
    st.session_state["messages"].append({"role": "user", "content":
user_input})

    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            response = requests.post(
                "http://localhost:8000/chat",
                json={
                    "message": user_input,
                    "history": st.session_state["messages"]
                }
            )
            data  = response.json()
            reply = data.get("response", "No reply received.")

            # Replace local history with the updated history returned by
the server
            st.session_state["messages"] = data.get("updated_history",
st.session_state["messages"])
            st.markdown(reply)
```

How Streamlit works

Streamlit re-executes the entire Python file from top to bottom every time the user interacts with the page. This sounds inefficient but is how Streamlit is designed — it's simple and predictable. `st.session_state` is a dictionary that persists across these re-runs, making it the right place to store the conversation history.

`st.chat_message("user")` renders a message bubble on the right with a user avatar. `st.chat_message("assistant")` renders a bubble on the left with a bot avatar. `st.spinner()` shows a loading indicator while the backend is processing.

Why store history on the frontend and not the backend?

This is a deliberate architectural choice. A stateless backend is simpler, cheaper, and scales horizontally — you can run multiple backend instances without any shared session storage. The tradeoff is that if the user refreshes the page, history is lost. For a production app, you'd persist history to a database (DynamoDB, PostgreSQL, etc.) and look it up by session ID.

---

9. End-to-End Request Flow

Let's trace exactly what happens when a user types: **"Show me all my S3 buckets"**

```
① frontend/chat.py
   User types "Show me all my S3 buckets"
   Message appended to st.session_state["messages"]

   HTTP POST → http://localhost:8000/chat
   Body: {
     "message": "Show me all my S3 buckets",
     "history": [{"role": "user", "content": "Show me all my S3 buckets"}]
   }

② backend/api/main.py
   FastAPI receives the POST at /chat
   Parses message and history from the request body
   Calls process_request(user_input, history)

③ backend/core/processor.py
   Instantiates AWSAgenticAgent()
   Calls agent.process_request("Show me all my S3 buckets", history)

④ backend/core/aws_agent.py — First Claude call
   Sends to Claude API:
     system:   SYSTEM_PROMPT
     messages: [{"role": "user", "content": "Show me all my S3 buckets"}]
     tools:    [aws_cloud_control definition, cloudwatch_logs definition]

   Claude responds with a tool_use block:
   {
     "type": "tool_use",
     "name": "aws_cloud_control",
     "input": {
       "operation": "list",
       "resource_type": "AWS::S3::Bucket"
     }
   }

⑤ backend/core/aws_agent.py — AWS API call
   _execute_aws_operation(operation="list",
resource_type="AWS::S3::Bucket")
   → boto3 cloudcontrol.list_resources(TypeName="AWS::S3::Bucket")
   → AWS returns:
```

```
   {
     "ResourceDescriptions": [
       {"Properties": "{\"BucketName\": \"my-photos\"}"},
       {"Properties": "{\"BucketName\": \"backups\"}"},
       {"Properties": "{\"BucketName\": \"static-assets\"}"}
     ]
   }

⑥ backend/core/aws_agent.py — Second Claude call
   _generate_summary(tool_name, raw_aws_result, "Show me all my S3
buckets")
   → Claude returns: "You have 3 S3 buckets: my-photos, backups, and
static-assets."

⑦ backend/api/main.py
   Returns HTTP 200:
   {
     "response": "You have 3 S3 buckets: my-photos, backups, and static-
assets.",
     "updated_history": [
       {"role": "user",      "content": "Show me all my S3 buckets"},
       {"role": "assistant", "content": "You have 3 S3 buckets: ..."}
     ]
   }

⑧ frontend/chat.py
   Renders the reply in the chat bubble
   Saves updated_history back to st.session_state
   UI is ready for the next message
```

---

10. Sample Interactions

These are real examples of how the chatbot behaves across different use cases. Each example shows the multi-turn nature of the agent — it asks for clarification when information is missing, executes the right AWS operation, and explains the result in plain English.

Interaction 1 — Querying CloudWatch Logs for Lambda Errors

This is the most common monitoring use case: checking if a deployed Lambda function has thrown errors recently.

```
You:  "Find out errors from my lambda function in the past 24 hours"

AI:   "I'd be happy to help you find errors from your Lambda function in
the past
       24 hours. However, I need one piece of information:

       What is the name of your Lambda function?
```

```
       Please provide the function name so I can query CloudWatch Logs for
errors
       from the past 24 hours."
```

> **What's happening here:** The user's request is missing the function name — a required parameter for the `cloudwatch_logs` tool. Because the system prompt explicitly says "NEVER assume missing required information", Claude asks rather than guessing. This prevents accidental queries against the wrong function.

```
You:  "ai_agent_demo"

AI:   📊 CloudWatch Logs Result: {"status": "success", "function_name":
"ai_agent_demo",
       "hours_back": 24, "error_count": 0, "error_logs": []}

       🤖 AI Summary:

       CloudWatch Logs Summary for ai_agent_demo

       Great news! I checked the CloudWatch logs for your ai_agent_demo
function
       over the last 24 hours, and everything looks healthy.

       Here's what I found:
       • No errors detected – The function ran without any issues (0
errors)
       • No error logs – There were no problematic log entries to report
       • Status: Success – Everything executed as expected

       This means your ai_agent_demo function is running smoothly!
```

> **What's happening here:** Claude calls the `cloudwatch_logs` tool with `function_name="ai_agent_demo"` and `hours_back=24`. The raw JSON result (`error_count: 0, error_logs: []`) is then passed to the summary prompt, and Claude translates it into a friendly health report. The raw JSON is shown briefly before the summary — in a production UI you'd hide it and show only the summary.

Interaction 2 — Creating an S3 Bucket (Multi-turn)

This shows the agent enforcing safe creation practices — always confirming the resource name before acting.

```
You:  "Create an S3 bucket for my photos"

AI:   "I'll help you create an S3 bucket for your photos. I need a few
details first:
```

```
        1. What would you like to name the bucket? (must be globally unique)
        2. What type of encryption would you prefer? (AES-256 or KMS)
        3. Should I enable versioning?
        4. Any specific access restrictions?"
```

> **What's happening here:** The system prompt contains an example of exactly this interaction. Claude is following the pattern it was given: identify missing info → ask for it → only then proceed. S3 bucket names must be globally unique across all AWS accounts, so generating a name automatically would be unreliable and potentially confusing.

```
You:  "my-photo-bucket-2024-sanjay, AES-256 encryption, no versioning,
private"

AI:   "Creating your S3 bucket now with the following configuration:
       - Name: my-photo-bucket-2024-sanjay
       - Encryption: AES-256 (SSE-S3)
       - Versioning: Disabled
       - Access: Private

       ✅ AI Summary: Your S3 bucket 'my-photo-bucket-2024-sanjay'
creation has been
       initiated successfully. The bucket is being set up with AES-256
encryption and
       private access. It should be available within a few seconds."
```

> **What's happening here:** Claude calls `aws_cloud_control` with `operation="create"`, `resource_type="AWS::S3::Bucket"`, and the properties gathered from your answers. Cloud Control returns a request token (the operation is async), and the summary prompt explains this in user-friendly terms.

Interaction 3 — Listing Existing Resources

```
You:  "Show me all my S3 buckets"

AI:   "🤖 AI Summary: You have 3 S3 buckets in your AWS account:
       • my-photo-bucket-2024-sanjay
       • data-backup-bucket
       • static-website-assets

       These are all the buckets currently in the us-east-1 region."
```

> **What's happening here:** No clarification is needed — listing S3 buckets requires no additional parameters. Claude immediately calls `aws_cloud_control` with `operation="list"` and `resource_type="AWS::S3::Bucket"`. The response is capped at 5 resources to keep the output manageable.

Interaction 4 — Conversational Follow-up (Memory in Action)

This shows how the conversation history enables follow-up questions without repeating context.

```
You:  "Show me all my S3 buckets"

AI:   "You have 3 S3 buckets: my-photo-bucket-2024-sanjay, data-backup-
bucket,
       and static-website-assets."

You:  "Tell me more about the backups one"

AI:   "Here are the details for data-backup-bucket:
       - Region: us-east-1
       - Versioning: Enabled
       - Encryption: AES-256
       - Created: 2024-03-15"
```

> **What's happening here:** "the backups one" is ambiguous on its own — but Claude has the full conversation history, so it knows this refers to `data-backup-bucket` from the previous message. Claude calls `aws_cloud_control` with `operation="read"` and `identifier="data-backup-bucket"`. This multi-turn awareness is what separates a chatbot from a single-shot AI query.

What These Interactions Teach You

Behavior observed	Why it happens
AI asks for the function name before querying	`NEVER assume missing required information` rule in the system prompt
AI asks for bucket name before creating	Same rule + explicit S3 example in the system prompt
Raw JSON appears before the AI summary	`_query_cloudwatch_logs` result is displayed, then `_generate_summary` adds the friendly version
"The backups one" is understood correctly	Full conversation history is sent to Claude on every request
Listing works immediately with no follow-up	`list` operation requires only `resource_type` — no identifier needed

---

11. Core AI Engineering Patterns

This project implements several patterns that appear in almost every production AI application. Understanding them will help you in any AI project you build next.

Pattern 1: Tool Use (Function Calling)

The foundation of AI agents. You define what functions exist (in `tools.json`), the AI decides which to call and with what arguments, and your code executes the actual function. The AI never directly runs code — it only describes *what to run*.

When to use it: Any time you want an AI to take real-world actions — querying a database, calling an API, sending an email, running a calculation.

Pattern 2: The System Prompt as a Contract

The system prompt (`SYSTEM_PROMPT` in `prompt.py`) is not just flavor text — it's a behavioral contract. Every rule you write there shapes how the AI responds in every conversation. Investing time in writing precise, explicit rules pays off in consistent, safe behavior.

Key techniques used here:

- Use "NEVER" / "ALWAYS" for hard constraints
- Provide explicit examples of correct behavior
- State what information must be confirmed before taking irreversible actions

Pattern 3: Two-Phase AI Calls (Reason → Respond)

The agent calls Claude twice per tool use:

1. **Reason:** Claude decides what action to take (which tool, which parameters)
2. **Respond:** Claude explains the result in plain English

Keeping these separate produces better results for both tasks. The first call focuses entirely on decision-making; the second focuses on communication. Mixing them would create conflicting optimization objectives in a single prompt.

Pattern 4: Conversation History as Context

There is no hidden memory system in this application. Conversational context is maintained by sending the full message history with every API call. This is how all current LLM applications work — the model has no persistent memory between calls unless you explicitly provide it.

The tradeoff: More history = better context, but also higher cost and latency (you pay per token, and longer inputs take longer to process). Production systems often implement *context compression* — summarizing old messages to reduce token count while preserving important information.

Pattern 5: Stateless Backend with Frontend State

The backend holds no session data. The frontend owns the conversation history and sends it with every request. This is the standard pattern for scalable AI applications — it keeps the backend simple and allows multiple instances to handle requests without coordination.

Pattern 6: Fast Model for Routine Tasks

Both AI calls in this project use `claude-haiku-4-5` — Anthropic's fastest model. Tool selection and result summarization are well-defined, constrained tasks that don't require the reasoning depth of a larger model. Matching model capability to task complexity is a core cost and latency optimization skill.

12. What to Learn Next

Now that you understand this project, here are the natural next steps on the AI engineering path:

Immediate Improvements to This Project

- **Streaming responses** — Instead of waiting for the full response, stream Claude's reply token by token using `client.messages.stream()`. This makes the UI feel much more responsive.
- **Persistent conversation history** — Store history in a database (DynamoDB works well since you're already on AWS) so conversations survive page refreshes.
- **Error handling for async AWS operations** — After a `create` call, poll `get_resource_request_status` until the resource is ready, then confirm to the user.

Broader AI Engineering Concepts

- **RAG (Retrieval-Augmented Generation)** — Instead of giving the AI all information in the prompt, retrieve relevant documents from a vector database and inject only what's needed. This scales to large knowledge bases.
- **Multi-agent systems** — Instead of one agent handling everything, build specialized agents (one for S3, one for EC2, one for billing) and an orchestrator that routes requests to the right agent.
- **Prompt evaluation and testing** — As your prompts grow in complexity, you need automated ways to test that the AI behaves correctly across many inputs. Libraries like PromptFoo and the Anthropic Evals framework help here.
- **Observability** — Log every Claude API call: the prompt sent, the response received, the tool selected, and the latency. Without this, debugging production issues is nearly impossible.

Cloud & Infrastructure

- **Deploy this app** — Package the FastAPI backend as a Docker container, deploy it on AWS ECS or Lambda. Deploy the Streamlit frontend on EC2 or use Streamlit Community Cloud.
- **IAM Roles over Access Keys** — In production, use IAM roles attached to EC2 instances or ECS tasks instead of static access keys. Roles are automatically rotated by AWS and can't be leaked the same way.

This documentation covers the complete application as built. For setup instructions, see README.md. For a quick file-level reference, see Agent.md.